

Participantes:

Juan David Reyes Cure
Juan Felipe Rodriguez Barbosa
Juan Esteban Rodriguez Castellanos

May 31, 2026

Entrenador:

Juan Jose Betancourt Lozano

Contents

1 Data Structures

1.1 Segment Tree

2 Geometry

2.1 Triangle Classification

3 Graph

3.1 Tree Degree Analysis

4 Math

4.1 Big Exponentiation
4.2 Big Exponentiation
4.3 Fibonacci Fast Doubling
4.4 Game Theory
4.5 GCD and LCM
4.6 Modular Arithmetic
4.7 Polynomial Composition

5 String

5.1 Partition and Permutation

6 Greedy

6.1 Budget Optimization
6.2 Segment Counting
6.3 Sorted Pairing

7 Implementation

7.1 Date Parsing

1 - Data Structures

1.1 Segment Tree

C++ (LeetCode) BlockPlacementQueries.cpp

Resuelve "Block Placement Queries": usa un segment tree de maximos y un set de obstaculos para determinar si un bloque de tamaño sz cabe en algun hueco del rango [0, x].

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef int T; // tipo de dato del segtree
5 struct segtree {
6     vector<T> st; //, lazy;
7     int n;
8     T neutro = 0; // neutro para MAX (los huecos son >= 0)
9
10    segtree(const vector<T> &v) {
11        n = v.size();
12        st.assign(n*4, 0);
13        //lazy.assign(n*4, neutro);
14        build(1, 0, n-1, v);
15    }
16
17    void build(int p, int L, int R, const vector<T> &v) {
18        if (L == R) st[p] = v[L];
19        else {
20            int m=(L+R)/2, l=p*2, r=l+1;
21            build(1,L,m,v); build(r,m+1,R,v);
22            st[p] = max(st[l], st[r]); // <- max
23        }
24    }
25
26    /*
27    void propagate(int p, int L, int R, T val) {
28        if (val == neutro) return;
29        st[p] = val;
30        lazy[p] = neutro;
31        if (L == R) return;
32        int l = p*2, r = l+1;
33        lazy[l] = lazy[r] = val;
34    }
35    */
36
37    T query(int i, int j) { return query(1, 0, n-1, i, j); }
38    void upd(int i, int j, T val) { upd(1, 0, n-1, i, j, val); }
39
40    T query(int p, int L, int R, int i, int j) {
41        //propagate(p, L, R, lazy[p]);
42        if (i > R || j < L) return neutro;
43        if (i <= L && R <= j) return st[p];
44        int m=(L+R)/2, l=p*2, r=l+1;
45        T lf = query(1, L, m, i, j);
46        T rg = query(r, m+1, R, i, j);
47        return max(lf, rg); // <- max
48    }
49
50    void upd(int p, int L, int R, int i, int j, T val) {
51        //propagate(p, L, R, lazy[p]);
52        if (i > R || j < L) return;
53        if (i <= L && R <= j) st[p] = val; // cambiar por propagate(p,L,R,val);
54        else {
55            int m=(L+R)/2, l=p*2, r=l+1;
56            upd(1, L, m, i, j, val);
57            upd(r, m+1, R, i, j, val);
58            st[p] = max(st[l], st[r]); // <- max
59        }
60    }
61 };
62
63 class Solution {
64 public:
65
66
67
```

```
68    vector<bool> getResults(vector<vector<int>>& queries) {
69        int MAX = 0;
70        for (auto &q : queries) MAX = max(MAX, q[1]);
71
72        vector<T> base(MAX + 1, 0); // árbol de huecos, todo en 0
73        segtree seg(base);
74
75        set<int> obs = {0}; // 0 es la pared izquierda
76        vector<bool> res;
77
78        for (auto &q : queries) {
79            if (q[0] == 1) { // colocar obstáculo en x
80                int x = q[1];
81                auto it = obs.lower_bound(x); // primer elemento > x
82                int prev = *std::prev(it);
83                seg.upd(x, x, x - prev); // hueco que termina en x
84                if (it != obs.end()) {
85                    int next = *it;
86                    seg.upd(next, next, next - x); // el hueco viejo se acorta
87                }
88                obs.insert(x);
89            } else { // consulta [2, x, sz]
90                int x = q[1], sz = q[2];
91                auto it = obs.upper_bound(x); // primer elemento > x
92                int prev = *std::prev(it); // mayor obstáculo <= x
93                bool ok = (x - prev >= sz) || (seg.query(0, x) >= sz);
94                res.push_back(ok);
95            }
96        }
97        return res;
98    }
99
100 }
101 };
```

2 - Geometry

2.1 Triangle Classification

C++ (RPG) VAROD.cpp

Clasifica triángulos en una grilla de p×q puntos como agudos, rectos, obtusos o degenerados. Utiliza análisis de productos para determinar el tipo de ángulo en cada triángulo.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 static inline int64 comb3(int64 n) {
7     if (n < 3) return 0;
8     return n * (n - 1) * (n - 2) / 6;
9 }
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int p, q;
16     cin >> p >> q;
17
18     if (p > q) swap(p, q); // p = lado pequeño, q = lado grande
19
20     const int W = p + 1; // cantidad de columnas de puntos
21     const int H = q + 1; // cantidad de filas de puntos
22
23     // gtProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) > k
24     // eqProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) = k
25     int maxS = p;
26     int maxK = maxS * maxS / 4;
27
28     vector<vector<int>> gtProd(maxS + 1, vector<int>(maxK + 1, 0));
29     vector<vector<int>> eqProd(maxS + 1, vector<int>(maxK + 1, 0));
30
31     for (int s = 0; s <= maxS; s++) {
32         int localMax = s * s / 4;
33         vector<int> freq(localMax + 1, 0);
```

```

34
35     for (int alpha = 0; alpha <= s; alpha++) {
36         int val = alpha * (s - alpha);
37         freq[val]++;
38     }
39
40     int suf = 0;
41     for (int k = localMax; k >= 0; k--) {
42         gtProd[s][k] = suf;
43         suf += freq[k];
44         eqProd[s][k] = freq[k];
45     }
46 }
47
48 auto getGt = [&](int s, int64 k) -> int64 {
49     int64 localMax = 1LL * s * s / 4;
50     if (k > localMax) return 0;
51     return gtProd[s][(int)k];
52 };
53
54 auto getEq = [&](int s, int64 k) -> int64 {
55     int64 localMax = 1LL * s * s / 4;
56     if (k > localMax) return 0;
57     return eqProd[s][(int)k];
58 };
59
60 int64 acute = 0, right = 0, obtuse = 0, degenerate = 0;
61
62 // -----
63 // 1) Todos en la misma fila -> degenerados
64 // -----
65 degenerate += 1LL * H * comb3(W);
66
67 // -----
68 // 2) Triángulos usando exactamente 2 filas
69 //
70 // Si la distancia vertical es h:
71 // - caso "punto solo entre la base"
72 // - caso "punto solo afuera de la base"
73 // -----
74 for (int h = 1; h <= q; h++) {
75     int64 verticalWays = 2LL * (H - h); // escoger las dos filas y cuál tiene 2 puntos
76
77     for (int s = 1; s <= p; s++) {
78         int64 waysPlaceX = W - s;
79         int64 weight = verticalWays * waysPlaceX;
80
81         // Punto solo entre los dos de la base
82         int64 rightBetween = 2 + getEq(s, 1LL * h * h); // extremos + posibles internos
83         int64 obtuseBetween = getGt(s, 1LL * h * h);
84         int64 acuteBetween = (s + 1) - rightBetween - obtuseBetween;
85
86         // Punto solo fuera del intervalo horizontal de la base
87         // todos esos son obtusos, salvo el caso alineado con un extremo,
88         // que ya quedó contado en "between" con alpha=0 o beta=0
89         int64 obtuseOutside = 2LL * (s - 1);
90
91         right += weight * rightBetween;
92         obtuse += weight * (obtuseBetween + obtuseOutside);
93         acute += weight * acuteBetween;
94     }
95 }
96
97 // -----
98 // 3) Triángulos usando exactamente 3 filas distintas
99 //
100 // Distancias verticales:
101 // fila1 -> fila2 = a
102 // fila2 -> fila3 = b
103 //
104 // Se cuenta por span horizontal total s.
105 // -----
106 for (int a = 1; a < q; a++) {
107     for (int b = 1; a + b <= q; b++) {
108         int64 verticalWays = H - (a + b);

```

```

09     int64 ab = 1LL * a * b;
10     int64 A = 1LL * a * (a + b);
11     int64 B = 1LL * b * (a + b);
12
13     for (int s = 0; s <= p; s++) {
14         int64 waysPlaceX = W - s;
15         int64 weight = verticalWays * waysPlaceX;
16
17         // Caso monótono:
18         // x1<=x2<=x3 o x1>=x2>=x3
19         // Todo no degenerado aquí es obtuso.
20         int64 orient = (s == 0 ? 1 : 2);
21
22         int64 degHit = 0;
23         if ((1LL * a * s) % (a + b) == 0) {
24             degHit = 1;
25         }
26
27         degenerate += weight * orient * degHit;
28         obtuse += weight * orient * ((s + 1) - degHit);
29
30         if (s == 0) continue;
31
32         // -----
33         // Caso donde x1 queda estrictamente "en medio"
34         // u < 0 y u+v > 0
35         // alpha en [1 .. s-1]
36         // -----
37         {
38             int64 total = s - 1;
39             if (total > 0) {
40                 int64 low = min<int64>(total, (ab - 1) / s);
41                 int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s - 1)) ? 1
42 : 0;
43
44                 int64 obt1 = low + getGt(s, A);
45                 int64 rig1 = rightLin + getEq(s, A);
46                 int64 acu1 = total - obt1 - rig1;
47
48                 obtuse += weight * 2 * obt1;
49                 right += weight * 2 * rig1;
50                 acute += weight * 2 * acu1;
51             }
52         }
53
54         // -----
55         // Caso donde x3 queda en medio o empata con x1
56         // v < 0 y u+v >= 0
57         // alpha en [0 .. s-1]
58         // -----
59         {
60             int64 total = s;
61             int64 low = min<int64>(total, (ab - 1) / s);
62             int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s)) ? 1 : 0;
63
64             int64 obt3 = low + getGt(s, B);
65             int64 rig3 = rightLin + getEq(s, B);
66             int64 acu3 = total - obt3 - rig3;
67
68             obtuse += weight * 2 * obt3;
69             right += weight * 2 * rig3;
70             acute += weight * 2 * acu3;
71         }
72     }
73 }
74
75 cout << acute << '\n';
76 cout << right << '\n';
77 cout << obtuse << '\n';
78 cout << degenerate << '\n';
79
80 return 0;
81 }

```

3 - Graph

3.1 Tree Degree Analysis

C++\Codeforces\comilos.cpp

Calcula cuántos pares de nodos pueden removerse de un árbol para desconectarlo. Es n menos la suma de los dos mayores grados de los nodos.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     vector<int> deg(n + 1, 0);
12
13     for (int i = 0; i < n - 1; i++) {
14         int a, b;
15         cin >> a >> b;
16
17         deg[a]++;
18         deg[b]++;
19     }
20
21     int mx1 = 0, mx2 = 0;
22
23     for (int i = 1; i <= n; i++) {
24         if (deg[i] >= mx1) {
25             mx2 = mx1;
26             mx1 = deg[i];
27         } else if (deg[i] > mx2) {
28             mx2 = deg[i];
29         }
30     }
31
32     cout << n - mx1 - mx2 << '\n';
33
34     return 0;
35 }
```

4 - Math

4.1 Big Exponentiation

C++\CCPL\qj.cpp

Calcula p^q usando aritmética de precisión arbitraria (boost::multiprecision) para manejar resultados con exponentes muy grandes.

```
1 #include <bits/stdc++.h>
2 #include <boost/multiprecision/cpp_int.hpp>
3
4 using namespace std;
5 using namespace boost::multiprecision;
6
7 int main() {
8     while (true) {
9         long long p, q;
10         cin >> p >> q;
11
12         if (p == 0 && q == 0) return 0;
13
14         cpp_int b = 1;
15
16         for (long long i = 0; i < q; i++) {
17             b *= p;
18         }
19
20         cout << b << "\n";
21     }
22 }
```

4.2 Big Exponentiation

JAVA\CCPL\qj.java

Calcula N^D usando BigInteger para manejar resultados con exponentes muy grandes.

```
1 import java.util.*;
2 import java.math.*;
3
4 class Main {
5     public static void main (String[] args) throws Exception {
6         Scanner sc = new Scanner(System.in);
7
8         while (sc.hasNext()) {
9             int N = sc.nextInt();
10             int D = sc.nextInt();
11
12             if (N == 0 && D == 0) break;
13
14             System.out.println(BigInteger.valueOf(N).pow(D));
15         }
16     }
17 }
```

4.3 Fibonacci Fast Doubling

C++\Codeforces\pou.cpp

Calcula el n -ésimo número de Fibonacci modulo 2026 usando exponenciación rápida de matriz.

```
1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 int fibmod(ll n, int m)
6 {
7     int a = 0, b = 1, c;
8     for (int i = 63-__builtin_clzll(n); i >= 0; i--)
9     {
10         c = a;
11         a = (1ll*c*(2ll*b-c+m)) % m;
12         b = (1ll*c*c + 1ll*b*b) % m;
13         if ((n>>i) & 1)
14             c = (a+b) % m; a = b; b = c;
15     }
16     return a;
17 }
18
19 int main() {
20
21     int n;
22     cin >> n;
23
24     cout << fibmod(n+1,2026) << endl;
25
26     return 0;
27 }
```

4.4 Game Theory

C++\Codeforces\digitalsiel.cpp

Determina el ganador de un juego analizando si $N+2$ es potencia de 2. Si lo es, el segundo jugador gana ("mastermei"), si no, el primero ("the greatest").

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPowerOfTwo(long long x) {
5     return x > 0 && (x & (x - 1)) == 0;
6 }
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    int T;
13    cin >> T;
14
15    while (T--) {
```

```

6      long long N;
7      cin >> N;
8
9      if (isPowerOfTwo(N + 2)) {
10         cout << "the greatest\n";
11     } else {
12         cout << "mastermei\n";
13     }
14 }
15
16 return 0;
17 }

```

4.5 GCD and LCM

C++\Codeforces\journeygrapes.cpp

Calcula a dividido entre mcd(a,b), que es el multiplicador para obtener mcm(a,b) desde b.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long a, b;
6     cin >> a >> b;
7     //c++17
8     cout << a / gcd(a, b) << endl;
9
10    return 0;
11 }

```

4.6 Modular Arithmetic

C++\CCPL\broul.cpp

Calcula el número mínimo de saltos de tamaño $3*b$ necesarios para cubrir la distancia vertical $abs(a-c)$, o determina si no es posible porque la distancia no es múltiplo de b.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     while(true){
7         int a,b,c;
8         cin >> a >> b >> c;
9         if(a==0 && b ==0 && c ==0)
10            {
11                return 0;
12            }
13
14         int god = abs(a-c);
15         if(god%b!=0)
16            {
17                cout << "No accounting tablet" << "\n";
18            }
19         else
20            {
21                if((god/b) % 3 ==0)
22                {
23                    cout << (god/b)/3<< "\n";
24                }
25                else
26                {
27
28                    cout << ((god/b)/3)+1<< "\n";
29                }
30            }
31
32
33    }
34
35
36
37
38    return 0;
39 }

```

4.7 Polynomial Composition

C++\Codeforces\coach.cpp

Calcula la composición de polinomios $P(Q(x))$ usando multiplicación polinómica modular. El resultado es modular 998244353 y tiene grado máximo $n*m$.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 998244353;
5
6 // Multiplica dos polinomios A y B
7 // A[i] representa el coeficiente de x^i
8 // B[j] representa el coeficiente de x^j
9 vector<long long> multiply(const vector<long long>& A, const vector<long long>& B) {
10     vector<long long> result(A.size() + B.size() - 1, 0);
11
12     for (int i = 0; i < (int)A.size(); i++) {
13         for (int j = 0; j < (int)B.size(); j++) {
14             // A[i] * x^i multiplicado por B[j] * x^j
15             // queda en el grado i + j
16             result[i + j] += A[i] * B[j];
17             result[i + j] %= MOD;
18         }
19     }
20
21     return result;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27
28     int n, m;
29     cin >> n >> m;
30
31     // P tiene grado n, por eso tiene n + 1 coeficientes
32     vector<long long> P(n + 1);
33
34     // Q tiene grado m, por eso tiene m + 1 coeficientes
35     vector<long long> Q(m + 1);
36
37     for (int i = 0; i <= n; i++) {
38         cin >> P[i];
39     }
40
41     for (int i = 0; i <= m; i++) {
42         cin >> Q[i];
43     }
44
45     // El grado máximo de P(Q(x)) es n * m
46     // Por eso la respuesta tiene n*m + 1 coeficientes
47     vector<long long> ans(n * m + 1, 0);
48
49     // power representa Q^i
50     // Al inicio i = 0, entonces Q^0 = 1
51     vector<long long> power = {1};
52
53     // Calculamos:
54     // P(Q(x)) = p0*Q^0 + p1*Q^1 + p2*Q^2 + ... + pn*Q^n
55     for (int i = 0; i <= n; i++) {
56
57         // Sumamos P[i] * Q^i a la respuesta
58         for (int j = 0; j < (int)power.size(); j++) {
59             ans[j] += P[i] * power[j];
60             ans[j] %= MOD;
61         }
62
63         // Preparamos la siguiente potencia:
64         // si ahora power = Q^i,
65         // después de multiplicar por Q queda power = Q^(i+1)
66         if (i < n) {
67             power = multiply(power, Q);
68         }
69     }
70 }

```

```

71 // Imprimimos los coeficientes r0, r1, ..., r(n*m)
72 for (int i = 0; i <= n * m; i++) {
73     cout << ans[i] << " ";
74 }
75
76 cout << "\n";
77
78 return 0;
79 }

```

5 - String

5.1 Partition and Permutation

C++\Codeforces\policia nacional.cpp

Divide una cadena en 3 partes, las reorganiza y verifica si forma otra cadena objetivo. Si es posible, imprime ambas descomposiciones ordenadas.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     string s1, s2;
12     cin >> s1 >> s2;
13
14     // Probar todos los cortes posibles:
15     // A = [0,i)
16     // B = [i,j)
17     // C = [j,n)
18
19     for(int i = 1; i < n-1; i++)
20     {
21         for(int j = i+1; j < n; j++)
22         {
23             string A = s1.substr(0, i);
24             string B = s1.substr(i, j-i);
25             string C = s1.substr(j);
26
27             vector<string> p = {A,B,C};
28
29             sort(p.begin(), p.end());
30
31             do
32             {
33                 string formed = p[0] + p[1] + p[2];
34
35                 if(formed == s2)
36                 {
37                     cout << "YES\n";
38
39                     // Descomposición original del pasaporte
40                     cout << A << " " << B << " " << C << "\n";
41
42                     // Descomposición de la visa
43                     cout << p[0] << " " << p[1] << " " << p[2] << "\n";
44
45                     return 0;
46                 }
47             } while(next_permutation(p.begin(), p.end()));
48         }
49     }
50
51     cout << "NO\n";
52
53     return 0;
54 }
55 }

```

6 - Greedy

6.1 Budget Optimization

C++\RPC\Posters.cpp

Dado un presupuesto c y costos a, b , calcula el maximo de posters que se pueden comprar comprando al menos uno de cada tipo.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     long long a,b,c;
8
9     cin >> a >> b >> c;
10
11     cout << (min((c-a) / b, (c-b) / a) < 0 ? 0 : min((c-a) / b, (c-b) / a)) << endl;
12
13
14
15     return 0;
16 }

```

6.2 Segment Counting

C++\Codeforces\cerros.cpp

Cuenta la cantidad de segmentos contiguos con elementos $\geq k$ en un arreglo. Cada vez que encuentra un elemento $\geq k$ después de uno $< k$, incrementa el contador.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     long long n,k;
9     cin >> n >> k;
10    long long god=0;
11    bool abierto = false;
12    for(int i = 0; i < n;i++)
13    {
14        int a; cin >> a;
15
16        if(a>=k&&!abierto)
17        {
18            god++;
19            abierto = true;
20        }
21        else if(a<k)
22        {
23            abierto = false;
24        }
25    }
26
27    cout << god << "\n";
28
29    return 0;
30 }

```

6.3 Sorted Pairing

C++\Codeforces\balatro.cpp

Calcula la suma de diferencias absolutas entre pares ordenados de dos arreglos. Ordena ambos y suma las diferencias elemento a elemento.

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6
7     long long n;
8     cin >> n;
9     vector<long long > yo(n);
10    vector<long long > atk(n);
11

```

```
12 for(int i = 0; i<n ; i++)
13 {
14     long long a;
15     cin >> a;
16     yo[i] = a;
17 }
18
19 for(int i = 0; i<n ; i++)
20 {
21     long long a;
22     cin >> a;
23     atk[i] = a;
24 }
25
26 long long ans = 0;
27
28 sort(yo.begin(), yo.end());
29 sort(atk.begin(), atk.end());
30
31 for(int i = 0; i<n ; i++)
32 {
33     ans+= abs(atk[i] - yo[i]);
34 }
35
36 cout << ans << endl;
37
38 return 0;
39 }
```

7 - Implementation

7.1 Date Parsing

C++11 RFC: <chrono>

(WIP) Lee dos fechas con d a de la semana y hora para calcular la diferencia de tiempo entre ellas.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     string d1,d2;
8     string h1,h2;
9     cin >> d1 >> h1 >> d2 >> h2;
10
11     string dias[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun", "Mon", "Tue", "Wed", "Thu", "
12     Fri", "Sat", "Sun"};
13
14
15
16     return 0;
17 }
```